Ders 24 Genişletilmiş Notu: Uygulama: POSIX Semaforları, Mesaj Kuyrukları, Paylaşılan Bellek ve Pthreads

Ders 24 Genişletilmiş Notu: Uygulama: POSIX Semaforları, Mesaj Kuyrukları, Paylaşılan Bellek ve Pthreads

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

Bu kayıt, İşletim Sistemleri (BLM3011) dersinin bir uygulama (laboratuvar) oturumudur. Oturumu dersin hocası Dr. Ziya Cihan Tayşi değil, bir öğretim elemanı anlatıyor. Slayt başlığı "İşletim Sistemleri Uygulama". Asıl anlatım yaklaşık 00:06:36'da başlar. Öncesi bekleme ve idari konuşmadır.

İçindekiler

- 1. Senkronizasyona Neden İhtiyaç Var?
- 2. Kaçınılan İki Durum: Açlık ve Ölümcül Kilitlenme
- 3. POSIX Semaforları
- 4. POSIX Mesaj Kuyrukları
- 5. Paylaşılan Bellek (Shared Memory)
- 6. Süreçler ve İş Parçacıkları
- 7. Pthreads ile İş Parçacığı Oluşturma ve Bekleme
- 8. Dört İş Parçacıklı Metin Örneği ve Yarış
- 9. Muteksler ve Koşul Değişkenleri
- Örnek Sorular
- Dersin Toparlaması

1. Senkronizasyona Neden İhtiyaç Var?

Öğretim elemanı semaforu, birçok süreç (process) tarafından paylaşılabilen bir veri yapısı olarak tanıttı. Bu yapılara, birden fazla sürecin aynı kaynağı ortak kullandığı problemlerde ihtiyaç duyarız. Kaynağın süreçlere senkronizasyon (synchronization) sağlanarak paylaştırılması gerekir.

Slayttaki şekilde dört süreç vardı ve hepsi aynı kaynak üzerinde değişiklik yapmak istiyordu. İki süreç aynı anda değişiklik yapmaya çalışırsa birinin yaptığı değişiklik diğerininkini etkileyebilir.

Örnek olarak ortak bir `counter` değişkeni düşünelim. P2 süreci değeri 2, P4 süreci de 1 artırıyor. İkisi de çalıştıktan sonra beklediğimiz şey, `counter` değerinin toplam 3 artmış olmasıdır. Ama ikisi aynı anda erişirse şu olabilir: P2 o anki değeri okuyup 2 artırırken P4 de aynı eski değeri okuyup 1 artırır ve sonucu yazar. Hangisi son yazdıysa onun etkisi kalır. Sonuçta değer 2 artmış da olabilir, 1 artmış da olabilir. İki süreç milisaniyeler arayla peş peşe çalışmışsa 3 artmış da olabilir. Bu duruma yarış durumu (race condition) denir.

Bunu önceden bilemeyiz, yani programın öngörülebilirliği azalır. Programımızın öngörülebilir olması için senkronizasyonu sağladığımızdan emin olmak isteriz. Semafor yapıları bunun için kullanılır.

2. Kaçınılan İki Durum: Açlık ve Ölümcül Kilitlenme

Kaynak paylaştırma problemini çözerken iki senaryodan uzak durulur: açlık (starvation) ve ölümcül kilitlenme (deadlock).

2.1 Açlık

Açlıkta süreç bir kaynağı bekler, ama kaynak ona bir türlü gelmez. Süreç "ölümüne bekler". Kaynağa erişemediği için işlemini hiçbir şekilde devam ettiremez.

2.2 Ölümcül Kilitlenme

Öğretim elemanı deadlock'u bir kavşak görseliyle anlattı. Dört araba bir kavşakta birbirinin yolunu kesmiş durumda. Arabalar yalnız düz ilerleyebiliyor, geri gidemiyor. Dördünün de yola devam edebilmesi için birinin ortadan kalkması gerekir. Ama hiçbiri kalkamaz.

Süreçler için durum şöyledir:

- P1'in beklediği kaynağı P2 tutar.
- P2'nin beklediği kaynağı P3 tutar.
- P3'ün beklediği kaynağı P4 tutar.
- P4'ün beklediği kaynağı P1 tutar.

Her süreç bir sonrakinin beklediği kaynağı tutar. Hiçbiri, beklediği kaynak gelmeden kendi kaynağını bırakmaz. Böylece hepsi birbirini sonsuza kadar bekler. Buna deadlock denir.

3. POSIX Semaforları

POSIX semaforları işletim sistemi düzeyinde oluşturulur. Slayttaki şekle göre bir semaforun iki alanı vardır:

- Bir tam sayı değer.
- Bekleyen süreçlerin kuyruğuna bir işaretçi. Başlangıçta bu işaretçi `NULL`'dır, yani hiçbir şeyi göstermez.

Semaforun tam sayı değeri hiçbir zaman sıfırın altına inmez.

3.1 Anahtar Benzetmesi

Öğretim elemanı semaforu bir anahtara benzetti. Bir süreç bir "kaynak odasına" girmek istediğinde semaforda anahtar varsa girer. Anahtar yoksa, anahtar gelene kadar dışarıda bekler. "Eksi anahtar" diye bir şey olamaz. Bu yüzden değer negatif olmaz. İşi bitince süreç anahtarı yerine bırakır.

3.2 İsimli Semaforlar

İsimli (named) semaforlar bir isimle oluşturulur ve bu isimle kullanılır. Beş fonksiyonu vardır:

- `sem_open`: Semafor ile süreç arasında bir bağlantı kurar. Semafor yoksa oluşturabilir.
- `sem_wait`: Semafordan anahtar ister. Değer sıfırdan büyükse değeri 1 azaltır ve devam eder. Değer sıfırsa, sıfırdan büyük olana kadar bekler, sonra 1 azaltır.
- `sem_post`: Tam tersini yapar. Paylaşılan kaynakla işi bitince anahtarı yerine bırakır, yani değeri 1 artırır.
- `sem_close`
- `sem_unlink`

Öğretim elemanı son iki fonksiyonu şöyle anlattı: `sem_close` semaforu komple sonlandırır, `sem_unlink` ise `sem_open` ile kurulan süreç-semafor ilişkisini kaldırır.

> **Ders sonrası not:** Öğretim elemanı `sem_close` ile `sem_unlink`'i ters anlattı. Doğrusu: `sem_close` yalnız çağıran sürecin isimli semaforla bağlantısını kapatır. `sem_unlink` semaforun adını sistemden siler. Semaforun kendisi, onu açık tutan son süreç de kapattığında yok olur.

3.3 İsimsiz Semaforlar

İsimsiz (unnamed) semaforların adı yoktur. Bu yüzden global olarak tanımlanırlar. Çalışacakları süreçlerle aynı bellek bölgesinde bulunmaları gerekir. Fonksiyonları aynı mantıktadır, yalnız `sem_unlink` yoktur ve iki fonksiyonun adı değişir:

- `sem_init`: Semaforu oluşturur ve ilk değerini verir.
- `sem_wait` / `sem_post`: İsimli semafordakiyle aynıdır. `wait` değeri 1 azaltır, `post` 1 artırır.
- `sem_destroy`: Semaforu ortadan kaldırır.

  Ek bilgi (derste anlatılmadı): `sem_init(&s, pshared, deger)` çağrısında `pshared = 0` ise semafor yalnız aynı sürecin iş parçacıkları arasında paylaşılır. Farklı süreçler arasında paylaşmak için `pshared` sıfırdan farklı verilir. Bu durumda semaforun kendisi de bir paylaşılan bellek bölgesine konmalıdır. "Aynı bellek bölgesinde olma" şartı bunu anlatır.

3.4 Örnek: İki Süreç, Bir Kritik Bölge

Slayttaki örnekte isimli bir semafor `sem_open` ile oluşturuldu ve adı `mutex` olan bir değişkene atandı. Açılamasaydı program hata verecekti. Semaforun ilk değeri 1 yapıldı.

A süreci şu döngüyü 10 kez çalıştırır:

- `sem_wait(mutex)` ile değeri 1 azaltır. Değer 1 olduğu için hemen girer.
- Kritik bölgeye (critical section) girdiği saati ekrana yazar: "Proses A kritik bölgeye girdi".
- Çıkış saatini yazar.
- `sem_post(mutex)` ile semaforu bırakır.
- 3 saniye bekler.

B süreci aynı işi kendi adıyla yapar.

Slayttaki çıktıda saatler okunaklı biçimde yazılmamıştı. Öğretim elemanı son basamaklara bakarak yorumladı. Örneğin A 54. saniyede kritik bölgeye girmiş ve yine 54'te çıkmış. Aynı 54. saniyede kritik bölgeye giren bir B görülmüyor. Çünkü A bitmeden B kritik bölgeye giremez. İki süreç aynı zaman diliminde kritik bölgede bulunamaz. Amaçlanan senkronizasyon budur.

Aynı yapı derleyip çalıştırılarak denendi (`gcc -Wall -pthread ... -lrt`): değeri 1 olan isimli semaforla A ve B süreçlerinin "girdi/çıktı" kayıtları hiçbir zaman iç içe geçmedi. Döngü sonunda semaforun değeri yine 1 oldu.

4. POSIX Mesaj Kuyrukları

4.1 Genel Özellikler

Mesaj kuyrukları (message queues), FIFO'lar (adlandırılmış borular) gibi kalıcı (persistent) nesnelerdir. Başta oluşturulmaları, artık gerekmediğinde de silinmeleri gerekir. Oluşturulurken iki sınır belirlenir:

- En fazla kaç mesaj tutulacağı.
- Her mesajın en büyük boyutu.

Mesaj kuyrukları `mq_open` sistem çağrısıyla oluşturulur ve açılır. Slaytta mesaj kuyruklarının sekiz fonksiyonu olduğu söylendi.

4.2 `mq_open` ve Bayraklar

`mq_open` parametrelerindeki bayraklardan derste şunlar geçti:

- `O_CREAT`: Kuyruk yoksa oluşturur.
- `O_EXCL`: Kuyruk zaten varsa `mq_open` hata verir.
- `O_NONBLOCK`: Çağrılar bloklanmaz, yani bekletilmez.

Kuyruğun adı `/` ile başlamalı ve başka `/` içermemelidir.

4.3 Kuyruk Öznitelikleri (`mq_attr`)

Kuyruk özniteliklerini tutan yapının dört alanı vardır:

- `mq_flags`: Az önceki bayraklar (ör. `O_NONBLOCK`).
- `mq_maxmsg`: Kuyruğun tutabileceği en fazla mesaj sayısı.
- `mq_msgsize`: Her mesajın en büyük boyutu.
- `mq_curmsgs`: Kuyrukta şu anda bulunan mesaj sayısı.

Öğretim elemanının örneği: 100 baytlık mesajları tutan ve 100 mesaj alabilen bir kuyruk düşünün. Burada `mq_maxmsg = 100`, `mq_msgsize = 100` olur.

4.4 `mq_close` ve `mq_unlink`

Öğretim elemanı `mq_close` için şunu söyledi: Bu çağrıdan sonra kuyruk komple silinmez. Kuyruktaki bütün mesajlar silinene kadar kuyruk kapatılmaz. Her kullanan `mq_close` çağırırsa, kuyrukta mesaj kalmadığında kuyruk silinir.

`mq_unlink` için de "silme işlemi kuyruktaki bütün mesajlar tüketilene kadar ertelenir" dedi. Slayttaki öneriye göre, kuyruğu kullanan bütün süreçlerin kuyruğu açtıktan hemen sonra `mq_unlink` çağırması genelde iyi bir uygulamadır. Böylece son süreç işini bitirince kuyruk kapanır.

> **Ders sonrası not:** Öğretim elemanı silme işleminin "mesajlar tükenene kadar" ertelendiğini söyledi; doğrusu farklı. `mq_close` kuyruğu hiç silmez, yalnız çağıran sürecin tanımlayıcısını kapatır. `mq_unlink` kuyruğun adını hemen siler. Kuyruğun kendisi, onu açık tutan bütün süreçler kapatınca yok olur. Bu, mesajların okunup okunmamasına bağlı değildir. Derlenerek denendi: içinde mesaj varken `mq_unlink` çağrıldıktan sonra kuyruk adla yeniden açılamadı, ama açık tanımlayıcıdan mesaj okunabildi.

4.5 Kalıcılık: Okuyucu ve Yazar Aynı Anda Açık Olmak Zorunda Değil

Mesaj kuyruğu kalıcıdır. FIFO'lardan farklı olarak içindeki mesajlar da kalıcıdır. Okuyucu ve yazarın kuyruğu aynı anda açması gerekmez. Bir yazar kuyruğu açabilir ya da oluşturabilir, mesajlar yazabilir, sonra kuyruğu kapatıp sonlanabilir. Daha sonra bir okuyucu kuyruğu açar ve mesajlar hâlâ oradadır, okuyabilir.

4.6 `mq_send`

`mq_send` parametreleri:

- `mqdes`: Mesajın gönderileceği kuyruğun tanımlayıcısı (message queue descriptor). Öğretim elemanı bunu "hedef kuyruk" olarak açıkladı.
- `msg_ptr`: Gönderilecek mesajın tamponunu (buffer) gösterir.
- `msg_len`: Mesajın uzunluğu. Kuyruğun tanımlı mesaj boyundan küçük ya da ona eşit olmalıdır, yoksa sığmaz.
- `msg_prio`: Mesajın önceliği (priority). Negatif olmayan bir sayıdır.

Kuyruk doluysa ve `O_NONBLOCK` bayrağı etkin değilse `mq_send`, kuyrukta yer açılana kadar bloklanır.

Derlenerek denendi: `mq_msgsize = 64` olan bir kuyruğa 100 baytlık mesaj gönderilince `mq_send` `-1` döndürdü (`EMSGSIZE`, "Message too long"). `O_EXCL` ile var olan kuyruğu ikinci kez oluşturma denemesi de `EEXIST` hatası verdi.

4.7 `mq_receive` ve Öncelik Örneği

`mq_receive` aynı parametreleri alır: kuyruk tanımlayıcısı, mesajın yazılacağı tampon, tamponun uzunluğu ve önceliğin yazılacağı değişken.

Slayttaki örnekte gönderici program çalıştırıldığında öncelikleri 11, 110 ve 17 olan üç ayrı mesaj gönderdi. Her mesaj "Hello from process …" metnini ve o anki tarihi içeriyordu. Alıcı programın çıktısında kuyruğun üç mesaj tuttuğu görüldü ("queue holds three messages"). Alıcı önce önceliği en yüksek olan, yani 110 öncelikli mesajı aldı ve yalnız onu ekrana yazdı.

Derlenerek denendi: aynı üç öncelikle gönderilen mesajlar art arda üç `mq_receive` çağrısında 110, 17, 11 sırasıyla alındı.

4.8 `fork` ve Mesaj Kuyrukları

Öğretim elemanı slayta dayanarak şunu söyledi: Açık bir mesaj kuyruğu, `fork` ile oluşturulan alt süreç tarafından devralınamaz. Bir alt süreç kuyruk tanımlayıcısı elde etmek için kuyruğu `mq_open` ile kendisi açmalıdır. Ayrıca "bir thread'i fork ile oluşturduğumuzda içindeki bütün elemanlar kopyalanır" dedi.

> **Ders sonrası not:** POSIX ve Linux'ta alt süreç, ebeveynin açık mesaj kuyruğu tanımlayıcılarının kopyalarını devralır (`man 2 fork`). Derlenerek denendi: `fork` sonrası alt süreç, ebeveynin tanımlayıcısıyla `mq_getattr` çağırabildi ve kuyrukta 3 mesaj gördü. Ayrıca `fork` thread değil, süreç oluşturur.

4.9 Boş Kuyruğu Fark Etmek ve `mq_notify`

`mq_receive` boş bir kuyruktan okumaya çalışırsa, mesaj gelene kadar bloklanır. Bu davranış istenmeyebilir. Bunu önlemek için kuyruğa `O_NONBLOCK` uygulanabilir. Bu durumda boş kuyrukta `mq_receive -1` döndürür. Süreç kuyruğun boş olduğunu anlar ve isteğini sonra tekrarlayabilir. Ne kadar gecikmeyle tekrarlanacağı programın ihtiyacına bağlıdır.

Ama bu yöntem işlemciyi meşgul eder. Öğretim elemanı şöyle örnekledi: Bir `while` döngüsü düşünün. "`-1` dönmeyene kadar sürekli kuyruktan okumaya çalış" diye yazdınız. İşlemci mesaj gelene kadar sürekli aynı şeyi dener ve ilerleyemez. Hem iş yapmaz, hem meşgul olur. Buna meşgul bekleme (busy waiting) denir.

Bunun yerine `mq_notify` kullanılır. Bu fonksiyon kuyruk ile süreci ilişkilendirir. Kuyruğa bir mesaj geldiğinde sürece bildirim gönderilir ve süreç uyandırılır. Kuyruğun durum değişikliğini iletmek için `sigevent` adlı bir yapı kullanılır.

- Bildirim bir kez gönderilir, sonra bildirim kaydı kaldırılır. Bir sonraki durum değişikliğinin de bildirilmesi için `mq_notify` yeniden çağrılmalıdır.
- Uyarılacak süreci değiştirmek için önce eski kayıt kaldırılır. Öğretim elemanının örneği: Kuyruğu P1'i uyaracak şekilde kurduk, şimdi P2'yi uyarmasını istiyoruz. Önce `mq_notify NULL` ile çağrılarak P1'in kaydı kaldırılır. Sonra P2 için ilişki baştaki gibi yeniden kurulur.

Derlenerek denendi: `O_NONBLOCK` verilmiş boş kuyrukta `mq_receive -1` döndürdü (`EAGAIN`).

4.10 Diğer Fonksiyonlar

- `mq_getattr` / `mq_setattr`: Öznitelikleri okur ve ayarlar. Öğretim elemanı bunların ayrıntısına girmedi; ihtiyaç oldukça öğrenilebilir dedi.
- `mq_timedsend` / `mq_timedreceive`: Gönderme ve almaya süre sınırı ekler. "Bir süre bekle, olmazsa iptal et" gibi çalışan yardımcı fonksiyonlardır.

5. Paylaşılan Bellek (Shared Memory)

5.1 Mesaj Kuyruğu ile Karşılaştırma

Paylaşılan bellek modelinde bir bellek bölgesi, işbirliği yapan süreçler tarafından paylaşılır. Slayttaki şekilde iki çözüm yan yana gösterildi:

- Mesaj kuyruğu çözümünde A ve B süreçleri bilgiyi kuyruk üzerinden alışveriş eder.
- Paylaşılan bellek çözümünde A ve B doğrudan aynı bellek alanına erişir.

Öğretim elemanına göre paylaşılan bellek, mesaj kuyruğundaki gibi birbirine mesaj gönderme (message passing) işlemi yapmaz. Süreçler aynı alana doğrudan erişir.

Asıl fark sistem çağrılarındadır. Mesaj kuyruğunda her gönderme ve alma bir sistem çağrısıdır. Bu sistemi daha çok meşgul eder. Paylaşılan bellekte sistem çağrısı yalnız alanı oluştururken kullanılır. Alan oluşturulduktan sonra bütün erişimler sıradan bellek erişimidir ve çekirdekten (kernel) yardım istenmez. Bu yüzden paylaşılan bellek, bilgisayar içinde bellek hızında iletişime izin verir ve mesaj kuyruğundan daha hızlıdır.

> **Ek bilgi (derste anlatılmadı):** Paylaşılan bellekte çekirdek erişimleri sıraya koymaz. İki süreç aynı alana aynı anda yazarsa 1. bölümdeki yarış durumu oluşur. Bu yüzden paylaşılan bellek genelde bir semaforla birlikte kullanılır.

5.2 Fonksiyonlar

Paylaşılan belleğin beş fonksiyonu vardır.

`shm_open` paylaşılan bellek nesnesini oluşturur ya da açar. Oluşturamazsa `-1` döndürür. Bayrakları:

- `O_CREAT`: Gerekirse yeni bir nesne oluşturur.
- `O_EXCL`: Nesne zaten varsa hata verir.
- `O_TRUNC`: Nesne zaten varsa boyunu sıfır bayta indirir.
- `O_RDWR`: Okuma ve yazma izni verir.
- `O_RDONLY`: Alanı yalnız okunabilir olarak açar.

`ftruncate` nesnenin boyunu `len` parametresine göre ayarlar. Nesne `len`'den büyükse fazlalık atılır. Kısaysa, `len` uzunluğuna gelene kadar sonuna içeriği sıfır olan baytlar eklenir.

`mmap` paylaşılan bellek nesnesinin (dosya tanımlayıcısı `fd` ile verilen) bir bölgesini sürecin adres alanına eşler. Eşleme, belirtilen ofsetten (offset) başlar ve verilen uzunluk kadar sürer. `addr` parametresi genelde 0 (`NULL`) verilir. Böylece bölgenin nereye eşleneceğine işletim sistemi karar verir. Gerekirse adres açıkça da verilebilir. `mmap` eşlediği alanın adresini döndürür.

Öğretim elemanı slayttaki şekil üzerinden şöyle özetledi: Süreç belli bir alana ihtiyaç duyar. `mmap` bu kadar alanı paylaşılan bellek nesnesinden alıp sürecin adres alanına eşler. Fonksiyonlar karmaşık görünse de her biri bir problemin çözümü olarak ortaya çıkmıştır.

`munmap` belirtilen adres aralığındaki eşlemeyi kaldırır, yani `mmap`'in tersini yapar. Öğretim elemanı bu fonksiyonun çok kullanılmadığını, yerine daha çok `shm_unlink` kullanıldığını, `munmap`'in "tedavülden kalktığını" söyledi.

> **Ders sonrası not:** `munmap` ile `shm_unlink` birbirinin yerine geçmez. `munmap` yalnız çağıran süreçteki eşlemeyi kaldırır. `shm_unlink` nesnenin adını hemen siler; nesne artık adla açılamaz. Nesnenin belleği, onu eşlemiş son süreç de `munmap` yapınca ya da sonlanınca serbest kalır. İkisi farklı işlerdir ve ikisi de güncel olarak kullanılır. Derlenerek denendi: `shm_unlink` sonrası eşlenmiş bellekteki veri hâlâ okunabildi, ama `shm_open` ile aynı ad açılamadı.

`shm_unlink` nesneyle olan bağlantıyı kaldırır. Nesneye başka bağlantılar varsa nesne var olmaya devam eder. `shm_unlink` çağıran süreç nesneyle ilişkili son süreçse, nesne silinir. Çünkü artık onu kullanan başka süreç yoktur.

5.3 Örnek

Slayttaki örnekte 56 baytlık bir paylaşılan bellek alanı oluşturuldu. Gönderici, alanın oluşturulduğunu ("shared memory segment allocated correctly") ve segment boyunu yazdı. Sonra alana bir mesaj yazdı. Mesajın türü (message type) ve bir sayısı rastgele atanmıştı. Öğretim elemanı programı daha önce çalıştırdığını ve her seferinde farklı değer geldiğini söyledi. Alıcı taraf aynı alandan bu mesaj türünü okudu.

Öğretim elemanına göre örnek, mesaj kuyruğu örneğinden çok farklı görünmüyor, çünkü yalnız bir mesaj gönderiliyor. Ama sistem tarafında bu şekilde yapılan paylaşım daha verimlidir.

Benzer bir akış derlenerek denendi: `shm_open` + `ftruncate(fd, 56)` sonrası nesnenin boyu 56 bayt oldu ve 56 baytın hepsi sıfırdı. `fork` ile oluşturulan alt sürecin eşlenmiş alana yazdığı metni ebeveyn okuyabildi.

6. Süreçler ve İş Parçacıkları

6.1 `fork` Neden Maliyetli?

Slayttaki şekilde A süreci `fork` ile B sürecini türetiyordu. Bu sırada A'nın global değişkenleri, kodu ve yığını (stack), yani bellek bölümlerinin hepsi B'ye kopyalanır. Bu çok maliyetlidir. Bu

yüzden iş parçacığı (thread) oluşturulur. Yeni bir thread oluşturmak, `fork` ile yeni süreç oluşturmaktan daha az maliyetlidir.

> **Ek bilgi (derste anlatılmadı):** Modern sistemlerde `fork` belleği hemen kopyalamaz. Yazmada kopyalama (copy-on-write) kullanılır: sayfalar ancak bir taraf yazdığında kopyalanır. Yine de yeni adres alanı ve sayfa tabloları kurulduğu için `fork`, thread oluşturmaktan daha maliyetlidir.

6.2 Thread'lerin Paylaştıkları ve Kendilerine Ait Olanlar

Bir süreç birçok thread içerebilir. Bir thread başka bir thread oluşturabilir, o da bir başkasını. Böylece bir sürecin birçok thread'i olabilir.

Aynı sürecin thread'leri şunları paylaşır:

- Bellek.
- Dosya ve soket tanımlayıcıları.
- Sinyal işleyicileri (signal handlers) ve sinyal ayarları.

Thread'ler bunlar üzerinden birbirleriyle bilgi alışverişi yapabilir.

Her thread'in kendine ait olanlar:

- Yığın (stack).
- Yazmaçlar (registers).
- Program sayacı (program counter).
- Kimlik (ID).

Öğretim elemanı burada iki kavramı hatırlattı. Yığın bir bellek bölgesidir, içinde bilgi tutulur. Program sayacı ise hangi satırın çalışacağını takip eder. Program sayacı hangi satırı gösteriyorsa o satır çalışır, sonra sayaç bir artırılır ve bir sonraki satır çalışır.

6.3 Paralel Programlamanın Zorluğu

Thread'lerin oluşturulması ve iş paylaşımı çok dikkatli yapılmalıdır. Tek bir `main` fonksiyonuyla program yazmaya alışmış bir programcı için paralel programlamaya geçmek zor olabilir.

Thread'lerle çalışırken derleyici hata vermez. "Şu satırda şunu yanlış yapmışsın" diyen bir gösterge yoktur. Programcı "önce bu satır çalıştı, sonra şu satır çalışacak" gibi sıralı düşünür. Ama paralel çalışan süreçlerle bir problemi çözmek zihni çok yoran bir iştir. Bu yüzden paralel programları yazmak ve hatalarını ayıklamak (debug) zordur. Öğretim elemanına göre paralel programlama insanı daha çok düşündürdüğü için daha çok geliştirir.

7. Pthreads ile İş Parçacığı Oluşturma ve Bekleme

7.1 `pthread_create`

```c
int pthread_create(pthread_t *tid, const pthread_attr_t *attr,
                   void *(*start_routine)(void *), void *arg);
```

`pthread_create` yeni bir thread oluşturur. Thread'in kimliği, işaretçi olarak verilen `tid` değişkenine yazılır. Her thread'in benzersiz (unique) bir kimliği vardır. Thread'lere bu kimlikle başvurulur.

Öğretim elemanı fonksiyonun başarılı olursa "sıfır değil" döndürdüğünü söyledi.

> **Ders sonrası not:** `pthread_create` başarılı olunca 0 döndürür. Hata durumunda sıfırdan farklı bir hata kodu döner. Derlenerek denendi: dönüş değeri 0 çıktı. Öğretim elemanı 7.4'teki `pthread_join` kontrolünü de "sıfırdan farklıysa sorun var" diye doğru anlattı.

7.2 Thread'e Argüman Göndermek

Thread fonksiyonuna yalnız bir argüman (`arg`) gönderilebilir. Birden fazla değer göndermek için değerler bir `struct` içine konur ve yapının adresi gönderilir. Slayttaki örnekte `x` ve `y` adında iki `int` alanı olan `foo` adlı bir yapı tanımlandı ve thread'e bu yapı gönderildi.

Öğretim elemanı slayta dayanarak bu yapıların yerel değişken (local variable) olamayacağını söyledi.

> **Ders sonrası not:** Bu kural tam doğru değildir. Asıl şart, yapının thread onu kullandığı sürece geçerli kalmasıdır. Yapı `main` içinde yerel tanımlanıp thread `pthread_join` ile beklenirse sorun olmaz. Tehlikeli olan, yapıyı tanımlayan fonksiyon thread bitmeden dönerse oluşur. Derlenerek denendi: `main` içindeki yerel `struct foo {3, 4}` thread'e verildi ve thread `x=3 y=4` yazdı.

7.3 Thread'in Ömrü

Program `main` fonksiyonunda başlar. `main` de bir thread'tir. Bu thread'in altında yeni thread'ler oluşturulabilir:

- `pthread_create` yeni thread'i oluşturur ve onu `start_routine` fonksiyonuyla başlatır.
- Thread, fonksiyonundan dönünce ya da `pthread_exit` çağırınca sonlanır.
- `main`, oluşturduğu thread'le birleşmek için `pthread_join` kullanır. Thread sonlanana kadar orada bekler.

Öğretim elemanı şekli şöyle anlattı: `main` thread'i yeni thread'i oluşturdu, sonra kendi yapması gereken işleri yaptı. Bu sırada yeni thread paralel çalıştı. `pthread_join` noktasında `main`, thread'in işini bitirip ana thread'e bağlanmasını bekler.

Slayttaki bir program dört thread oluşturur, her birini çalıştırır ve kendisi sonlanmadan önce hepsinin sonlanmasını bekler.

7.4 "Hello" Örneği

Slayttaki örnekte bir `hello` fonksiyonu vardı. Parametre alır, thread'in numarasını basarak "hello" yazdırır, sonra `pthread_exit` ile thread'i sonlandırır.

- Fonksiyon `void *` türünde bir argüman alır. Önce bu argümanı `int` türüne dönüştürür ve böylece thread numarasını elde eder.
- `main`, `NUM_THREADS` (ör. 5) boyunda bir thread kimliği dizisi ve her thread'in argümanını tutan bir argüman dizisi oluşturur.
- `pthread_attr_init` ile öznitelik nesnesine varsayılan değerler okunur.
- Döngüde `pthread_create` çağrılır: birinci parametre thread kimliği, ikinci öznitelikler, üçüncü `hello` fonksiyonu, dördüncü i. thread'in argümanı.
- Sonra her thread için `pthread_join` çağrılır. `pthread_join` belirtilen kimlikteki thread sonlanana kadar bekler. Dönüş değeri sıfırdan farklıysa bir sorun olduğu anlaşılır. Sıfırsa hata bloğuna girilmez ve program mesajını yazar.

Öğretim elemanı fonksiyonların "`void` dönüş türüne sahip olması gerektiğini" söyledi.

> **Ders sonrası not:** Thread fonksiyonu `void` değil, `void *` döndürmelidir. Fonksiyon sonucu `return` ile ya da `pthread_exit` ile bir işaretçi olarak verir.

Öğretim elemanı `pthread_join`'i bir yerde "birinin bitmesi, diğerinin başlaması" diye de özetledi.

> **Ders sonrası not:** Doğru anlam, slayttaki "wait for thread with thread ID" ifadesidir: çağıran thread, belirtilen thread sonlanana kadar bekler. `pthread_join` başka bir thread'i başlatmaz.

7.5 Joinable ve Detached Thread'ler

Thread'ler iki türlü oluşturulabilir:

- Birleştirilebilir (joinable): İşi bitince `main`'e katılabilir, yani `pthread_join` ile beklenebilir.

- Ayrık (detached): Hiçbir şekilde `main`'e geri dönüş yapamaz, `join` edilemez.

Ayrık thread oluşturmak için öznitelik nesnesinde `pthread_attr_setdetachstate` fonksiyonu kullanılır.

Derlenerek denendi: `PTHREAD_CREATE_DETACHED` ile oluşturulan thread için `pthread_join` 22 (`EINVAL`) döndürdü.

8. Dört İş Parçacıklı Metin Örneği ve Yarış

8.1 Fonksiyonlar

Bu örnekte aynı global metin üzerinde dört thread aynı anda çalışır:

- Birincisi metnin uzunluğunu hesaplar.
- İkincisi metindeki boşlukları sayar.
- Üçüncüsü metnin bütün harflerini büyük harfe çevirir.
- Dördüncüsü bütün harfleri küçük harfe çevirir.

Hepsi `void *` döndürür ve bir argüman alır. Boşluk sayan fonksiyon argümanı karakter dizisine dönüştürür, çünkü harflere tek tek erişmek ister. Bir işaretçiyle karakterleri gezer: karakter boşluğa eşitse sayacı 1 artırır, sonra bir sonraki karaktere geçer. Sonunda sonucu global `number of space` değişkenine yazar. Uzunluk ve boşluk sayısı global değişkenlerde tutulur.

8.2 Step 1 ve Step 2

Slaytta iki çözüm vardı.

- **Step 1:** `NUM_THREADS` 4'tür. Dört elemanlı bir kimlik dizisi oluşturulur, öznitelikler varsayılan değerlerle alınır. Ama her thread farklı bir fonksiyon çalıştırdığı için `pthread_create` satırı dört kez yazılır: önce uzunluk fonksiyonuyla, sonra boşluk sayan fonksiyonla, sonra büyük harf, sonra küçük harf fonksiyonuyla.
- **Step 2:** Dört fonksiyon bir diziye (fonksiyon işaretçisi dizisi) konur. Thread kimlikleri bir dizide tutulduğu gibi fonksiyonlar da bir dizide tutulur ve i indisine bağlanır. Böylece thread'ler tek bir döngüyle, tek satırla oluşturulur. Öğretim elemanı bunu çok güzel ve etkili bir çözüm olarak niteledi.

8.3 Sonuçlar

Slayttaki çıktılarda:

- Uzunluk her seferinde 36 çıktı.
- Boşluk sayısı her seferinde 5 çıktı.
- Büyük ve küçük harf thread'leri ise birbiriyle yarıştı. Bir çalıştırmada metnin yarısı büyük, yarısı küçük çıktı. Başka birinde büyük harf thread'i daha hızlı çalışıp yarışı kazandı ve metnin hepsi büyük çıktı. Bir başkasında neredeyse hepsi küçük çıktı, yalnız birkaç harf büyük kaldı.

Uzunluk ve boşluk hesapları metni değiştirmediği için birbirini etkilemez; 36 ve 5 her çalıştırmada aynıdır. Ama metni aynı anda hem büyük hem küçük harfe çevirmeye çalışan iki thread sonucu öngörülemez yapar. Hangi thread'in önce çalışacağına biz karar vermedikçe bunu bilemeyiz. Hangisi o anda işlemciyi kullanabiliyorsa o daha hızlı ilerler ve sonucu etkiler.

Öğretim elemanının benzetmesi: Bir ile biri topladığımda her zaman 2 görmek isterim. Çoğunlukla 2, ama bazen 1, bazen 3 gören bir sonucu kabul etmemiz mümkün değildir.

9. Muteksler ve Koşul Değişkenleri

9.1 Paylaşılan Global Değişken Problemi

Önceki örnekte global bir metne bütün thread'ler aynı anda erişiyordu. Biri büyük, biri küçük harfe çevirmeye çalışıyordu. Paylaşılan global değişkenlerdeki bu problemler muteksler (mutex) ve koşul değişkenleri (condition variables) ile çözülebilir.

9.2 Karşılıklı Dışlama ve Muteks

Bu problemlerden kaçmanın bir yolu karşılıklı dışlamadır (mutual exclusion). Bir global değişkene erişmeden önce onu kilitlemek, işi bitince kilidi açmak demektir.

Öğretim elemanının benzetmesi: Tuvaleti kullanıyorum. Herkesin girip çıkmasını istemediğim için girdikten sonra kapıyı kilitliyorum. İşim bitince kapıyı açıp çıkıyorum. Ben içerideyken dışarıdan kimse o odayı kullanamaz.

Muteksin iki temel fonksiyonu vardır:

- `pthread_mutex_lock`: Kilidi alır. Kilit başka bir thread'deyse, kilit açılana kadar bekler.
- `pthread_mutex_unlock`: Kilidi bırakır.

Öğretim elemanı, slayta dayanarak, bu fonksiyonların hepsinin bloklama yapabildiğini ve muteks değişkenlerinin statik olması gerektiğini söyledi.

> **Ders sonrası not:** Bloklayan çağrı `pthread_mutex_lock`'tur; `pthread_mutex_unlock` beklemez. "Muteks değişkenleri statik olmalı" ifadesi yalnız `PTHREAD_MUTEX_INITIALIZER` ile ilklendirme için geçerlidir; bu makro statik (ya da global) değişkenlerde kullanılır. Dinamik olarak ayrılan muteksler `pthread_mutex_init` ile ilklendirilir.

9.3 Koşul Değişkenleri

Muteksle çözülen durumlarda bir sorun kalır. Odanın kapısı kilitli. Ben her seferinde kalkıp kapıyı açmaya çalışıyorum. Kilitli, içeride hâlâ biri var. Gidiyorum, sonra tekrar geliyorum, yine deniyorum. Bunu sürekli yapmak işlemciyi yorar. Bu meşgul beklemedir.

Koşul değişkeni şuna benzer: "Tuvalet boşalınca zil çalsın." Zili duyduğumda tuvaletin boşaldığını anlar ve giderim. Koşul değişkenleri de meşgul bekleme problemini ortadan kaldırır. Bekleyen thread uyur. Verilen koşul sağlandığında, örneğin paylaşılan bir değer beklenen değere geldiğinde, thread uyandırılır.

Slaytta bu fonksiyonlar gösterildi, ama altyazıda adları okunmuyor.

> **Ek bilgi (derste anlatılmadı):** Temel iki fonksiyon `pthread_cond_wait` (koşul sağlanana kadar uyu) ve `pthread_cond_signal` (bekleyen bir thread'i uyandır) fonksiyonlarıdır. `pthread_cond_wait` her zaman bir muteksle birlikte çağrılır. Çağrı muteksi bırakıp uyur, uyanınca muteksi yeniden alır. Koşul bir `while` döngüsünde kontrol edilir, çünkü thread sahte uyanma (spurious wakeup) ile koşul sağlanmadan da uyanabilir. Derlenerek denendi: bir thread `while (!hazir) pthread_cond_wait(...)` ile bekledi, ana thread `hazir = 1` yapıp `pthread_cond_signal` çağırınca uyandı.

Örnek Sorular

Derste ekran görüntüsü yoktu. Aşağıdaki sorular öğretim elemanının anlatım sırasında sözlü sorup cevapladığı sorulardır. Öğrenciler akademik bir soru sormadı.

Soru 1 (derste sözlü, 00:07:51): Ortak bir `counter` değişkenini P2 süreci 2, P4 süreci 1 artırıyor. İkisi de çalıştıktan sonra `counter` ne kadar artmış olmalıdır? Senkronizasyon yoksa ne olabilir?

> **Çözüm (derste verildi):** Beklenen artış $2 + 1 = 3$'tür. Senkronizasyon yoksa ikisi aynı eski değeri okuyup üzerine yazabilir. O zaman son yazanın etkisi kalır ve değer yalnız 2 ya da yalnız 1 artmış olur. Peş peşe çalışırlarsa 3 artar. Sonuç öngörülemez.

Soru 2 (derste sözlü, 00:26:29): `mq_receive` boş bir kuyruktan mesaj okumaya çalışırsa ne olur?

> **Çözüm (derste verildi):** `O_NONBLOCK` yoksa çağrı mesaj gelene kadar bloklanır. `O_NONBLOCK` varsa `-1` döner. Bunu döngüde tekrar denemek meşgul beklemedir. Bunun yerine `mq_notify` ile mesaj gelince bildirim almak tercih edilir.

Soru 3 (derste sözlü, 00:41:03–00:41:13): Her thread'in kendine ait bir yığını ve program sayacı vardır. Yığından ve program sayacından ne anlıyoruz?

Çözüm (derste verildi): Yığın, içinde bilgi tutulan bir bellek bölgesidir. Program sayacı hangi satırın çalışacağını takip eder: gösterdiği satır çalışır, sonra sayaç bir artırılır ve bir sonraki satır çalışır.

Soru 4 (derste sözlü, 00:45:50–00:51:51): `pthread_join` çağrıldığında ne olur?

Çözüm (derste verildi): Çağıran thread (ör. `main`), kimliği verilen thread sonlanana kadar bekler. Dönüş değeri 0 ise birleşme başarılıdır; sıfırdan farklıysa bir hata vardır.

Dersin Toparlaması

Bu uygulama oturumunda, paralel çalışan süreç ve thread'lerin paylaşılan kaynakları neden senkronize kullanması gerektiği gösterildi. Önce yarış durumu, açlık ve ölümcül kilitlenme tanıtıldı. Sonra POSIX semaforlarının isimli ve isimsiz türleri, `sem_wait`/`sem_post` ile kritik bölgeye giriş-çıkış ve iki sürecin kritik bölgede hiç aynı anda bulunmadığı örnek görüldü.

Süreçler arası iletişim için iki yol karşılaştırıldı. Mesaj kuyruklarında her işlem bir sistem çağrısıdır, mesajlar önceliğe göre alınır ve `mq_notify` meşgul beklemeyi önler. Paylaşılan bellekte sistem çağrısı yalnız kurulumda gerekir, bu yüzden daha hızlıdır. Son olarak pthreads ile thread oluşturma, `join`/`detach` ve dört thread'li metin örneğindeki yarış görüldü. Bu yarışın çözümü olarak muteksler ve meşgul beklemeyi kaldıran koşul değişkenleri tanıtıldı. Öğretim elemanına göre dersin sonunda paylaşılan birimlerin neden senkronize kullanılması gerektiği ve koşul değişkenlerinin neden kullanıldığı anlaşılmış olmalıdır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.